

DIAL Terminal 3 International Departures

A Dynamic Stocks and Flows Model of Passenger Arrival, Check-in Queueing, and Counter Assignment

Prepared for Delhi International Airport Limited

15 June 2026

Contents

1	Scope and reading guide	1
2	Glossary	3
3	Entities and their attributes	4
3.1	The passenger i	4
3.2	The flight f	4
3.3	The counter j	5
3.4	The service group	5
4	Constraints and conventions	6
5	Governing equations (stocks and flows)	7
6	The engine: time, schedule, and arrival laws	8
7	The engine: classes, counters, and policies	8
8	The engine: scenario assembly	8
9	Lanes, classes, and the home-departure margin	9
10	Solved cases	10
11	Verification	11
12	What is variable: the full set of levers	12
13	Agenda for the leadership specification meeting	13
14	Appendix: notation summary	14

1 Scope and reading guide

This document specifies and implements a single, fully parameterised model of the passenger check-in process for international departures from Terminal 3 (T3) at Delhi airport. Everything in scope here is T3 international only. The model is written as a stocks and flows system.¹

¹A stocks and flows system tracks quantities that accumulate over time (stocks, for example the number of people currently waiting in a queue) and the rates that change them (flows, for example people joining the queue and people being processed). It

Every assumption is isolated, named, and changeable. The same engine runs a zero-assumption ideal case and progressively richer cases by swapping one configuration object. The text uses mathematical notation for precision; footnotes translate technical terms into plain language where useful.

The order of the document is: glossary, entity definitions, notation, governing equations, the configurable engine, solved cases, verification, and a closing agenda of items to confirm with leadership.

is the standard way to describe a queue that fills and drains over time.

2 Glossary

The following terms are used with fixed meanings throughout.

Term	Meaning
SOBT	Scheduled Off-Block Time. The scheduled departure datum, written T .
Check-in window	The interval during which a flight accepts check-in, fixed at $[T - 4h, T - 1h]$.
Open / Close	Check-in opens at $T - 4h$ and closes at $T - 1h$. These are fixed.
Show-up	A passenger who actually arrives to check in for the flight.
Entry	The instant a passenger joins the check-in queue, written e_i .
Service begin	The instant a counter starts processing a passenger, written b_i .
Processed	The instant a passenger finishes check-in, written d_i .
Lag	Time from entry to processed, $\ell_i = d_i - e_i$.
Wait	Time from entry to service begin, $w_i = b_i - e_i$.
No-show	A passenger who arrives after Close. Only possible once the after-Close tail is allowed.
Left-behind	A passenger who entered before Close but was still queued at Close.
GHA	Ground Handling Agent. The firm staffing the counters (for example AISATS, BFS).
Counter / Desk	One check-in position. One counter equals one staff member.
Lane	A class restriction on a counter, for example a business-only desk.
Load factor	Fraction of seats that become show-ups, written ρ (a variable assumption).
Class	A service segment a passenger belongs to, written κ_i (economy, premium, business, first, fast-track, loyalty).
Δt	The model time step in minutes. Variable, default 1.
Idiosyncrasy	A rule that makes one group differ from the base case (by airline, destination, or departure time).

3 Entities and their attributes

We define four entities: the passenger, the flight, the counter, and the service group. Each is a row in a table whose columns are the attributes below. All symbols are reused consistently for the rest of the document.

3.1 The passenger i

A passenger i carries the following attributes.

Symbol	Attribute	Meaning
i	id	Unique passenger identifier.
$f(i)$	flight	The flight this passenger is booked on.
$a(i)$	airline	Airline of the flight, $a(i) = a_{f(i)}$.
$r(i)$	route	Destination routing of the flight.
$g(i)$	GHA	Ground handler of the flight, $g(i) = g_{f(i)}$.
κ_i	class	Service class or lane membership (see Section 8).
L_i	loyalty	Indicator, 1 if loyalty-priority, else 0.
X_i	fast-track	Indicator, 1 if fast-track product held, else 0.
e_i	entry time	Minute at which the passenger joins the queue.
b_i	service begin	Minute a counter starts processing the passenger.
d_i	processed time	Minute the passenger finishes check-in.
w_i	wait	$w_i = b_i - e_i$.
ℓ_i	lag	$\ell_i = d_i - e_i$.
m_i	margin	Buffer minutes the passenger targets before Close (Section 9).
s_i	status	One of served, no-show, left-behind.

3.2 The flight f

Symbol	Attribute	Meaning
φ_f	designator	Flight number, for example AI 173.
a_f	airline	Operating carrier code.
r_f	route	Destination routing.
S_f	SOBT	Scheduled off-block time in minutes from a reference midnight.
H_f	hour	Clock hour of S_f .
K_f	seats	Physical seat capacity.
ρ	load factor	Show-up rate, variable assumption.
N_f	show-ups	$N_f = \text{round}(\rho K_f)$. Area under the arrival curve.

Symbol	Attribute	Meaning
g_f	GHA	Assigned ground handler.
o_f	open	$o_f = S_f - 240$ minutes.
c_f	close	$c_f = S_f - 60$ minutes.
$\mathcal{D}_f$	arrival law	Arrival distribution family and parameters (Section 7).
$n_{f,\kappa}$	class split	Show-ups in class κ , with $\sum_{\kappa} n_{f,\kappa} = N_f$.

3.3 The counter j

Symbol	Attribute	Meaning
j	id	Unique counter identifier.
g_j	GHA	Ground handler operating the counter.
a_j	airline	Airline the counter is assigned to serve.
μ_j	rate	Service rate in passengers per minute. Service time per passenger is $1/\mu_j$.
o_j	open	Minute the counter is staffed from.
c_j	close	Minute the counter stops, bounded by the served flights' Close.
$\mathcal{A}_j$	lane	Set of classes the counter may serve.

3.4 The service group

Counters are pooled by ground handler and then by airline, not by flight.² A service group is the key (g, a) . Every passenger with $g(i) = g$ and $a(i) = a$ shares the counter pool of group (g, a) . Setting the group key to (g, a, f) recovers a per-flight model, which the engine also supports.

²This matches operations: a handler staffs a bank of desks for an airline, and any of that airline's departing passengers in the window can use any of those desks.

4 Constraints and conventions

The model enforces the following.

1. Fixed window. Check-in opens at $o_f = S_f - 240$ and closes at $c_f = S_f - 60$ for every flight. These are not adjustable.
2. Mass conservation. The area under each flight's arrival curve equals its show-ups:

$$\int_{o_f}^{c_f} \lambda_f(t) dt = N_f, \quad N_f = \text{round}(\rho K_f),$$

where $\lambda_f(t)$ is the arrival intensity (passengers per minute) for flight f .³ When the after-Close tail is allowed, the integral runs beyond c_f and the excess mass becomes no-shows.

3. Per-passenger deadline. A passenger can only be processed up to his own flight's Close. If $b_i \geq c_{f(i)}$ the passenger is left-behind.
4. Resource budget. Counters are scarce. The total counter-minutes consumed, $\sum_j (c_j - o_j)$, must not exceed an operating budget B . Adding counters shortens the time each can be assigned elsewhere.
5. Ideal feasibility. The ideal constraint is zero left-behind and zero no-show. A scenario that violates it is feasible to run but flagged.

³Intensity is how many passengers arrive per minute at time t . Its total area across the window is the number of people who show up.

5 Governing equations (stocks and flows)

Fix a service group and a class lane. Let $Q(t)$ be the queue stock, the number of passengers who have entered but not yet begun service at time t . Let $\alpha(t)$ be the arrival flow into the queue and $\delta(t)$ the departure flow out of the queue into service. The stock evolves as

$$Q(t + \Delta t) = Q(t) + \alpha(t) \Delta t - \delta(t) \Delta t, \quad Q(t) \geq 0.$$

The departure flow is capped by the available service capacity of the open counters serving that lane:

$$\delta(t) \Delta t = \min\left(Q(t) + \alpha(t) \Delta t, \sum_{j \in \text{open}(t)} \mu_j \Delta t\right).$$

The arrival flow is the discretised arrival law, $\alpha(t) \Delta t = N_f \cdot \int_t^{t+\Delta t} \hat{\lambda}_f(u) du$, where $\hat{\lambda}_f$ is the normalised arrival density. Conservation across the whole window returns $\sum_t \alpha(t) \Delta t = N_f$ by construction.

The per-passenger lag is read off the service schedule: $\ell_i = d_i - e_i$ and $w_i = b_i - e_i$. The engine resolves the order of service inside each step so that these are defined for every individual, which is the quantity the planner ultimately cares about.

6 The engine: time, schedule, and arrival laws

The arrival-law registry is the first swappable block. `uniform` is the base case. `normal` takes a mean and standard deviation. `linear` takes an intercept and a gradient. `skew` is the complex family, a skew-normal with location, scale, and shape. `data` reproduces the 35/40/25 split given in the source workbook, which is retained only as a reference case and is, as a model of true arrivals, a weak assumption.⁴

7 The engine: classes, counters, and policies

The simulator is exact for the priority and lane rules: it advances only to instants where something can change (an arrival, a counter freeing, a counter opening, or a deadline), and at each such instant gives every free desk the best eligible passenger it is allowed to serve. The queue stock $Q(t)$ is then read back from entry and service-begin times for any time grid the planner chooses.

8 The engine: scenario assembly

⁴The source split assigns fixed shares to three one-hour blocks and says nothing about within-hour timing, so it cannot represent build-up or rush behaviour. It is included for comparison, not as a recommended law.

9 Lanes, classes, and the home-departure margin

The class segmentation in Section 8 doubles as the lane structure. Under the **shared** lane policy every counter serves every class and the priority order alone decides who is taken first. Under the **dedicated** policy counters are split across classes in proportion to class load, with any spare desks left shared.⁵

Class is also the natural carrier of the home-departure margin. A passenger targets an arrival time and leaves home earlier by a travel time drawn from an isodistance or isochrone radius around the airport, adjusted for traffic and time of day.⁶ Formally, for passenger i with target arrival t_i^* and travel time τ_i ,

$$h_i = t_i^* - \tau_i, \quad e_i = t_i^* + \varepsilon_i,$$

where h_i is the leave-home time and ε_i is small arrival noise. Higher classes tend to choose a larger margin, so t_i^* shifts earlier for them. The current build draws e_i directly from the arrival law; the margin path above is the hook through which the isochrone work will feed e_i once the catchment data is available, without changing anything downstream.

⁵Dedicated lanes can lower the wait for premium passengers while raising it for economy, because capacity is reserved. The model reports both so the trade is visible.

⁶An isodistance radius groups origins by straight-line distance; an isochrone groups them by travel time, which is the more useful notion in traffic. A traffic and time adjusted isochrone stretches that travel time by the congestion expected at the relevant hour.

10 Solved cases

We run two cases through the identical engine and compare. Case S0 is the zero-assumption ideal: uniform arrivals, one lane, all desks open at $T - 4$, equal service speed, six desks per group. Case S-data swaps in the workbook's 35/40/25 arrival split and nothing else, isolating the effect of that one assumption.

Table 5: Case comparison through one shared engine.

case	passengers served		left_behind	no_show	mean_wait	p90_wait	max_wait	mean_lag	counter_min
S0 uniform (ideal target)	5842	4243	1599	0	21.76	93.45	150.75	24.26	15428
S-data 35/40/25 split	5842	4243	1599	0	19.46	82.00	140.00	21.96	15425

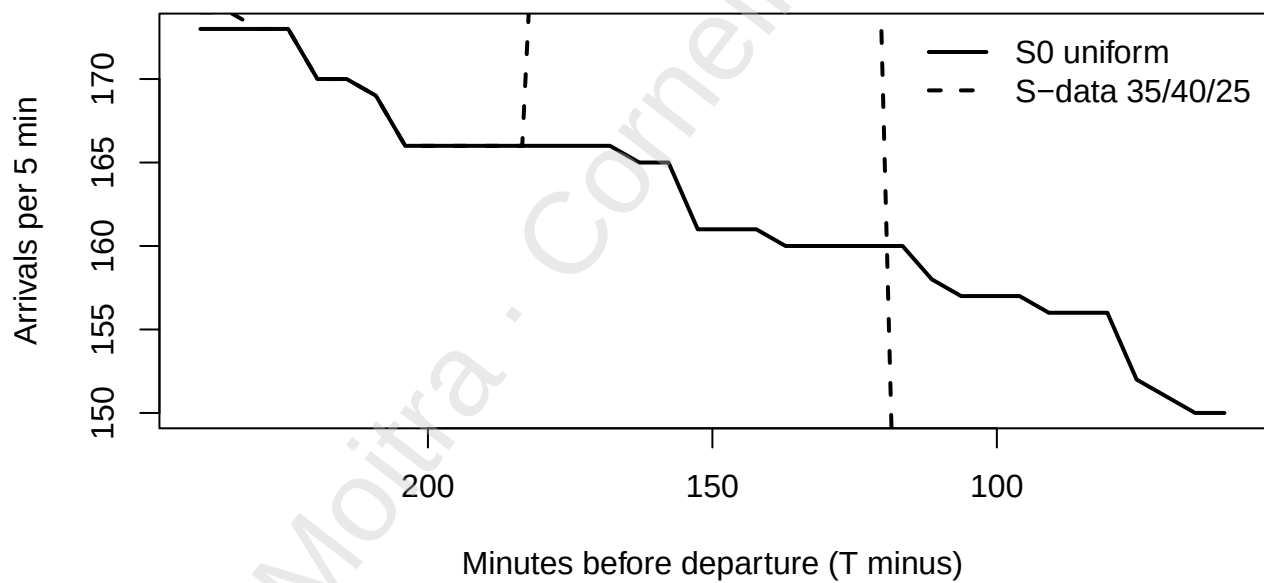


Figure 1: Arrival intensity per 5 minutes: uniform base versus the workbook split, summed across all flights on a common pre-departure axis.

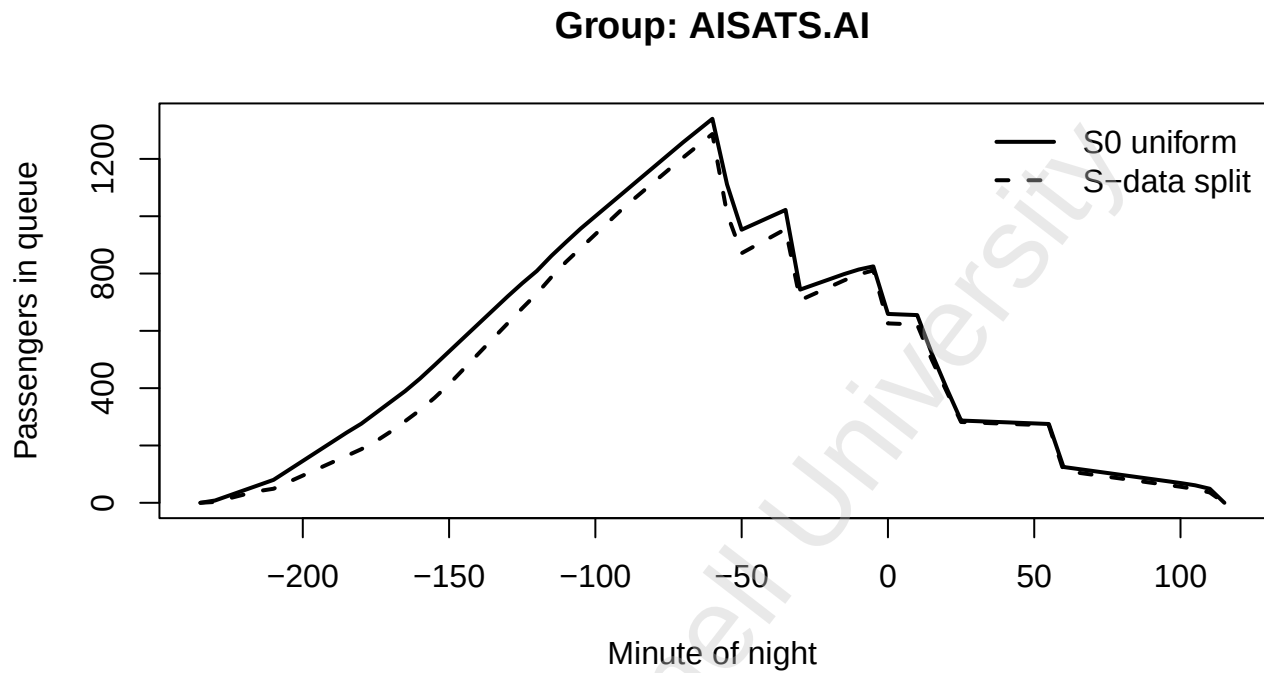


Figure 2: Queue stock over time for the busiest service group under each case.

11 Verification

The checks below run on every knit. They confirm mass conservation (arrivals equal show-ups), that the engine respects the fixed window, and that the ideal constraint is reported rather than silently violated.

Table 6: Automatic verification, recomputed on each knit.

check	result
Arrivals equal show-ups (all flights)	PASS
All entries within [open, close]	PASS
No passenger served after own close	PASS
dt divides 180	PASS

Ideal constraint (zero left-behind, zero no-show) under S0: VIOLATED, see KPI table

12 What is variable: the full set of levers

Every item below is a single field in the scenario list and can be changed without touching the engine.

Lever	Field	Options or range
Time step Δt	<code>dt</code>	any divisor of 180 (1, 2, 3, 5, 6, 10, ...)
Load factor ρ	<code>load_factor</code>	0 to 1
Arrival law	<code>arrival\$family</code>	uniform, normal, linear, skew, data
Arrival parameters	<code>arrival\$params</code>	mean, sd, intercept, gradient, xi, omega, alpha, mass
Idiosyncrasy	<code>arrival_by</code>	overrides by airline, route, or hour
Class structure	<code>segment</code>	uni, bi, bi_ff, tri, quad, quint
Lane policy	<code>lane_pol</code>	shared, dedicated
Opening policy	<code>open_pol</code>	all_T4, planned, responsive
Planned stagger	<code>planned_steps</code>	minute offsets after group open
Service rate μ	<code>mu</code>	passengers per minute
Counters per group	<code>counters_per_group</code>	the decision variable, typically 5 to 7
Grouping	<code>group_key</code>	c("gha", "airline") or add "fid" for per-flight

To run a new case, copy `default_scenario()`, change the fields of interest, and call `run_scenario(flights, sc)`. The counter count is the primary decision variable; sweeping it against the resource budget B is the optimisation that this engine is built to support next.

13 Agenda for the leadership specification meeting

The model is built to be steered by a small number of decisions that only operations leadership can fix. The following should be confirmed before the model is calibrated and used to set rosters.

1. Service rate. What is the realistic passenger-per-minute rate per desk for international check-in, and does it differ by ground handler.
2. Counter budget. The total counters and counter-hours available per night per handler, which sets the resource budget B and the feasible range for the counters-per-group decision.
3. Arrival law of record. Which arrival family should be the planning default, and whether the workbook 35/40/25 split is to be retired.
4. Class and lane policy. The cabin and product mix to model per airline, and whether dedicated lanes are used for premium, fast-track, or loyalty.
5. Service standard. The target wait, for example a p90 wait ceiling, and the tolerance for left-behind passengers, which defines the ideal constraint in practice.
6. Catchment data. Access to travel-time or isochrone data around the airport so the home-departure margin can be calibrated rather than assumed.
7. Idiosyncrasies that matter. Which airlines, destinations, or hours leadership believes deviate enough to warrant their own arrival law.

14 Appendix: notation summary

Symbol	Meaning
T	SOBT, the departure datum
o_f, c_f	flight open ($T - 4\text{h}$) and close ($T - 1\text{h}$)
Δt	time step in minutes
ρ	load factor
K_f, N_f	seats and show-ups, $N_f = \text{round}(\rho K_f)$
$\lambda_f, \hat{\lambda}_f$	arrival intensity and its normalised density
κ_i	passenger class or lane
e_i, b_i, d_i	entry, service begin, processed times
w_i, ℓ_i	wait and lag
$Q(t)$	queue stock
$\alpha(t), \delta(t)$	arrival and departure flows
μ_j	service rate of counter j
$o_j, c_j, \mathcal{A}_j$	counter open, close, and allowed lanes
B	counter-minute budget

This document is generated from `dial_t3_checkin_model.Rmd`. Knit to PDF to reproduce every number and figure above.